

Session 07 — Project Guide

TaskFlow: Lists + Derived Data

This guide documents the official **Session 07 End** state of TaskFlow. Everything in it comes from the real project files — nothing is invented, and it does not include homework solutions. Keep this guide — you can read it at home, days later, and rebuild the whole mental model from it alone.

1. The Session Story — From Session 06 to Session 07

Session 06 gave you real state — filter buttons highlighted, search remembered what you typed, a task list could hide and show. But one thing never changed no matter what you clicked or typed: the three task rows themselves. They were hardcoded, written by hand, three separate times.

Session 07 fixes exactly that gap. Today, the task list becomes real — built *from* an array of data instead of typed out by hand — and, for the first time, clicking "Completed" or typing a search term genuinely changes which rows are visible.

The shift in one sentence: **you stop writing UI for specific tasks, and start writing UI for whatever tasks the data happens to contain.**

2. Where Today's Project Actually Came From

Before any of today's new code, it's worth understanding what project you started from, because it isn't simply "Session 06 End."

Starting at this exact boundary, the permanent rule for this course is:

$$\text{Session N+1 Start} = \text{Session N End} + \text{Session N's FULL homework solution}$$

"FULL" means the complete CORE + STRETCH + CHALLENGE solution — never a CORE-only one — so that whatever any individual student personally wrote, everyone ends up on the same canonical baseline: `SectionTitle` (with a required `subtitle`), `PersonSummary`, a small name-and-greeting feature, and its own Show/Hide toggle, all beyond bare Session 06 End.

That's the intended rule, and it's the one that matters going forward. Tonight specifically, a delivery mix-up meant the project actually opened in class matched bare Session 06 End instead — the full homework solution above hadn't made it into the classroom copy yet. That gap has

since been fixed, and what you have now **is** the correct, complete Session 07 state described in this guide. The rule itself stands: from here on, the full CORE + STRETCH + CHALLENGE solution is what becomes the next session's starting point, regardless of which tiers you personally completed.

3. Sample Data vs. State

This is the single most important distinction in today's session, so it comes first, before any code.

Two different kinds of information exist in this project now:

- **Sample data** — `tasks` and `users`, plain `const` arrays. They are read constantly, but nothing in this session ever changes what's inside them.
- **State** — `currentFilter`, `searchText`, `selectedUserId`. Each of these genuinely changes while the app runs, and React needs to know when they do.

```
const tasks: Task[] = [  
  { id: 1, userId: 1, title: "Finish JavaScript exercise", completed: false  
},  
  { id: 2, userId: 2, title: "Review pull request", completed: true },  
  // ...four more  
];
```

`tasks` sits **outside** the `App` function entirely, declared once, at module level — exactly like a constant you'd declare in any other TypeScript file. Nothing about `useState` is involved.

Why does this distinction matter today specifically? Because it would be easy to assume "the task list is now dynamic, so `tasks` must be state." It's the opposite: `tasks` stays a plain array *because* nothing yet changes it. The moment a feature needs to add, remove, or edit a task (Session 08), `tasks` will become genuine React state — but not before there's a real reason.

4. `Task / User` Interfaces — Already Familiar

```
interface Task {  
  id: number;  
  userId: number;  
  title: string;  
  completed: boolean;  
}
```

```
interface User {  
  id: number;  
  name: string;  
}
```

If this looks familiar, it should — this is the exact same `Task / User` shape from the TypeScript sessions, before React existed in this course. Nothing about `interface` is new today. What's new is only that this data now lives inside a React project, describing sample arrays instead of API responses.

5. What `.map()` Solves

Here's the actual problem Session 06 left open: three `TaskItem` calls, written by hand, one per task. That worked when there were exactly three tasks and they never changed. It stops working the moment there are six tasks, or sixty, or a number that changes.

The familiar idea: you already know `for (const task of tasks) { ... }` — visiting every item in an array, one at a time.

The new idea: `.map()` does the same visiting, but instead of just running code for each item, it *collects* what each visit produces into a brand-new array.

```
tasks.map(function (task) {  
  return (  
    <TaskItem  
      key={task.id}  
      title={task.title}  
      ownerName={getOwnerName(task.userId)}  
      statusText={task.completed ? "Completed" : "Pending"}  
      statusClass={task.completed ? "completed" : "pending"}  
    />  
  );  
});
```

Read it exactly as: "for every `task` in `tasks`, produce one `TaskItem`, and collect all of those `TaskItem`s into a new list." `.map()` never changes `tasks` itself — it produces a separate, new array of React elements sitting right there in your JSX.

6. The Normal-Function Callback Form

The code above uses a **normal function** as the callback — the exact kind of function you've written since Session 1: `function (task) { ... }`. Nothing about this is React-specific; `.map()` simply calls this function once for every item, handing it that one item as `task`, and using whatever it `return`s as that item's place in the new array.

This is the actual, real first version of Session 07's list rendering — not a simplified example. It works completely correctly exactly as written.

7. The Arrow-Function Form — And Why It's the Same Thing

Immediately after building the version above, it gets rewritten into a shorter syntax:

```
tasks.map((task) => {
  return (
    <TaskItem
      key={task.id}
      title={task.title}
      ownerName={getOwnerName(task.userId)}
      statusText={task.completed ? "Completed" : "Pending"}
      statusClass={task.completed ? "completed" : "pending"}
    />
  );
});
```

Compare the two function shapes directly:

```
function (task) { return ...; }    // normal function
(task) => { return ...; }          // arrow function
```

These do the exact same thing. An arrow function is simply a shorter way to write a function — commonly used with array methods (`.map()`, `.filter()`) and throughout React generally. It is not a different kind of function with different behavior; it's different punctuation around the identical idea. From this point forward, new callback-style code in this project uses arrow functions — but `getOwnerName`'s own helper (Section 10) intentionally stays a normal function, because nothing requires rewriting code that already works.

8. `key` — Giving Each Item a Stable Identity

Notice `key={task.id}` in both versions above. This is not optional decoration — React specifically requires it whenever you render a list built from an array.

Why it exists: when React re-renders a list, it needs to tell "this is the same item, still here" apart from "this is a genuinely new item." Without a stable identity, React can only guess based on position in the list — which becomes wrong the moment items can ever be reordered, added, or removed.

Why `task.id` specifically is a good key:

- It's **unique** — no two tasks share one.
- It's **stable** — it never changes for a given task.
- It **already belongs to the task** — nothing was invented just to satisfy React.

`key` goes on the outermost element `.map()` returns for each item (the `<TaskItem>` tag itself), never inside the component's own list of props.

9. Mapping `Task` → `TaskItem` — The Complete Walkthrough

The real, final rendering, from `App.tsx` :

```
<ul className="task-list">
  {visibleTasks.map((task) => {
    const statusText = task.completed ? "Completed" : "Pending";
    const statusClass = task.completed ? "completed" : "pending";

    return (
      <TaskItem
        key={task.id}
        title={task.title}
        ownerName={getOwnerName(task.userId)}
        statusText={statusText}
        statusClass={statusClass}
      />
    );
  })}
</ul>
```

`TaskItem` itself did not change at all this session — it's the identical component from Session 05, still reading `props.title / props.ownerName / props.statusText / props.statusClass`. What changed is only *how many times* it gets called, and *where the props come from*: instead of four typed-in string literals, each prop is now computed fresh from one real `task` object as `.map()` visits it.

10. Deriving the Owner Name

`TaskItem` needs an owner's *name* (`"Leanne Graham"`), but a `Task` only stores a `userId (1)`. Connecting the two reuses `.find()`, already taught in the TypeScript sessions:

```
function getOwnerName(userId: number): string {
  const user = users.find(function (user) {
    return user.id === userId;
  });

  if (user) {
    return user.name;
  }

  return "Unknown person";
}
```

`.find()` searches `users` and returns the first match, or `undefined` if nothing matches — so the `if (user)` check is the same null-guard discipline you've used since the TypeScript sessions, applied here to a genuinely possible "no such person" case. This function is deliberately left as a normal function, not rewritten into an arrow — it already worked correctly before arrows existed in this course, and there's no reason to touch it.

11. What "Derived Data" Means

This is the single biggest idea of the whole session, so it's worth stating as its own rule, separate from any one example:

If a value can be calculated from data and state that already exist, calculate it during rendering — don't create another `useState` for it.

A derived value is never "remembered" anywhere. It's recomputed, fresh, every single time the component runs. That means it can never go stale, never be forgotten, and never disagree with the data it came from — because it's never stored separately from that data in the first place.

`visibleTasks` , `totalCount` , `completedCount` , `pendingCount` , each person's task count, and "X of Y tasks shown" are **all** derived. None of them are `useState` .

12. `visibleTasks` — The Complete Walkthrough

```
const search = searchText.toLowerCase();

const visibleTasks = tasks.filter((task) => {
  let matchesFilter = false;

  if (currentFilter === "all") {
    matchesFilter = true;
  } else if (currentFilter === "completed" && task.completed) {
    matchesFilter = true;
  } else if (currentFilter === "pending" && !task.completed) {
    matchesFilter = true;
  }

  const title = task.title.toLowerCase();
  const matchesSearch = title.includes(search);

  let matchesPerson = false;

  if (selectedUserId === 0) {
    matchesPerson = true;
  } else if (task.userId === selectedUserId) {
    matchesPerson = true;
  }

  return matchesFilter && matchesSearch && matchesPerson;
});
```

Read this top to bottom, once, out loud: for every task, check whether it matches the current status filter; separately check whether its title contains the search text; separately check whether it belongs to the selected person. Only if **all three** are true does the task survive into `visibleTasks` .

13. Reusing `.filter()`

`.filter()` is not new — it's the exact tool already used in the TypeScript sessions to build a smaller array from a bigger one, keeping only the items where the callback returns `true`. Nothing about its behavior changed; only the callback is now an arrow function instead of a normal one, since this code was written after arrows were introduced (Section 7).

14. Combining Status + Search + Person

`.toLowerCase()` / `.includes()` are reused directly from the TypeScript sessions' own search logic — lower-casing both sides makes the match case-insensitive, and `.includes()` checks for a substring anywhere in the title. Nothing here is new syntax.

The three checks — `matchesFilter`, `matchesSearch`, `matchesPerson` — are each computed completely independently of one another inside the same callback. They only come together on the very last line, `return matchesFilter && matchesSearch && matchesPerson;`. This means the *logic* is already fully prepared to combine all three — status, search, and person — the moment all three can actually be set from the UI. Right now, two of them can: searching for "review" while "Completed" is selected already narrows the list correctly, and "All people" (`selectedUserId === 0`) already participates too. What you **can't** do yet is pick one specific named person through the UI — the named buttons that would set `selectedUserId` to a real person's id aren't wired yet (Section 19). `visibleTasks` doesn't need any changes once they are; the filtering logic is already there, waiting.

15. Why `visibleTasks` Is NOT State

It would be tempting to think "the visible list changes, so it must be state." But `visibleTasks` never needs to be *told* to change — it's recalculated automatically every time `App` re-renders, using whatever `tasks`, `currentFilter`, `searchText`, and `selectedUserId` currently hold. Making it `useState` would mean manually remembering to recompute and re-set it every time any of those four things changed — reintroducing exactly the "forgot to update it" bug class that state setters exist to prevent.

16. Dynamic Stats

```
const totalCount = tasks.length;

const completedCount = tasks.reduce(function (count, task) {
```



```

    if (task.completed) {
      return count + 1;
    }

    return count;
  }, 0);

const pendingCount = totalCount - completedCount;

```

`totalCount` always reads the full sample size — six tasks — regardless of any filter. `completedCount` and `pendingCount` are also derived, never hardcoded, so they stay correct automatically no matter what the sample data contains.

17. Reusing `.reduce()`

`.reduce()` is reused directly from the TypeScript sessions: it carries one running value (`count`) from task to task, adding one each time `task.completed` is true, and starting from `0` . This `function (count, task) { ... }` callback is left as a normal function to match how it was originally taught — nothing requires rewriting it into an arrow.

Below the task list:

```

<p className="visible-count">{visibleTasks.length} of {tasks.length} tasks
shown</p>
<p className="progress">{completedCount} of {totalCount} tasks completed</p>

```

"X of Y tasks shown" answers a different question than the stats above it — it tells you how many of the *full* sample are currently visible after filtering, using `visibleTasks.length` against `tasks.length` . Type something into search and watch it change in real time.

18. People Counts — Real, Derived

The old, hardcoded `PersonSummary` calls are gone. In their place:

```

{users.map((user) => {
  const personTaskCount = tasks.filter((task) => task.userId ===
user.id).length;

  if (personTaskCount === 0) {

```

```

        return null;
    }

    return (
        <PersonSummary key={user.id} name={user.name} taskCount=
{personTaskCount} />
    );
  })}

```

For every person, `.filter()` keeps only the tasks belonging to them, and `.length` counts how many survived. `PersonSummary` itself is completely unchanged from Session 06's homework — it still just renders `props.name / props.taskCount`. Only *what's passed in* changed: a real, calculated number instead of a number someone typed by hand. The `personTaskCount === 0` guard means a person with no loaded tasks is skipped entirely rather than showing "0 tasks" — a small defensive habit, not a new concept.

19. `selectedUserId` and the Person-Filter Buttons — "All People" Works, Named Buttons Don't Yet

```
const [selectedUserId, setSelectedUserId] = useState(0);
```

`0` means "All people" — a plain number, the exact scalar `useState` shape already known since Session 06, just holding a number this time. You already saw in Section 12 that `visibleTasks` knows how to use it: when `selectedUserId` is `0`, every person matches; otherwise only tasks belonging to that one person do.

```
function handleShowAllPeople() {
  setSelectedUserId(0);
}
```

```

<section className="filters">
  <button
    className={"filter-button" + (selectedUserId === 0 ? " active" : "")}
    onClick={handleShowAllPeople}
  >
    All people
  </button>

```

```
{users.map((user) => (  
  <button key={user.id} className="filter-button">  
    {user.name}  
  </button>  
))}  
</section>
```

`handleShowAllPeople` is a plain, no-argument named handler, exactly like every filter handler this session — clicking "All people" genuinely resets `selectedUserId` to `0` right now, and because `visibleTasks` already reads `selectedUserId`, the rendered task list and "X of Y tasks shown" both respond correctly the moment you click it. **The three stat cards do not** — Total/Completed/ Pending are derived from the full `tasks` array (Section 16), not `visibleTasks`, so they stay 6/3/3 no matter which person, status, or search is selected. Only "X of Y" reacts to filtering.

The named buttons are intentionally unfinished at this point. Each one already renders with the correct name and a real, stable `key={user.id}` — that part came from the same `.map()` you learned earlier today. But notice what's missing compared to "All people": no `onClick`, and no active-class ternary. Clicking a person's name right now does nothing at all.

Completing this is this session's homework, not something built today. Think about what you'd need: a handler that knows *which* person was clicked (not just "reset to zero" like `handleShowAllPeople` does), a way to give that handler the right person's id when a specific button is clicked, and an active-class expression like the one already used for "All people," adapted to compare against each button's own person. Every one of those ideas has already appeared somewhere in this guide or in Session 06 — homework is about combining them, not learning something new.

20. `type` and the `FilterStatus` Union

```
type FilterStatus = "all" | "completed" | "pending";
```

`type` gives a reusable name to a TypeScript type — the same idea `interface` already gives you, a different keyword, that can also name something that isn't an object shape. Here, it names a closed set of exactly three allowed strings. Try assigning `setCurrentFilter("done")` anywhere and TypeScript rejects it — this union is what makes that rejection possible.

```
const [currentFilter, setCurrentFilter] = useState<FilterStatus>("all");
```

The `<FilterStatus>` part tells TypeScript exactly what values this particular state is allowed to hold — the same idea as any other type annotation you've written, applied to `useState`'s own generic slot. Nothing deeper about generic types is needed to use this correctly today.

21. What Happens When Nothing Matches — A Known, Unsolved Gap

Try it: pick a search term that matches no task title, or combine filters until nothing survives into `visibleTasks`. Right now, the task list area simply renders **nothing at all** — no rows, no message, just empty space where the list used to be. It can easily look like the app is broken, even though it's working exactly as designed: `visibleTasks` correctly became an empty array, and `.map()` over an empty array correctly produces nothing to render.

This is a real, known gap in tonight's End state, and it's intentionally left unsolved — it's this session's STRETCH homework. You already have every piece you need to fix it:

`visibleTasks.length` tells you exactly when this happens, and Session 06 already taught you the exact conditional- rendering shape (check a condition, render one thing or another) that solves it. An `.empty-state` CSS class already exists in the stylesheet, instructor-provided and currently unused, ready for whatever element the message ends up on — so styling isn't the exercise, wiring the behavior is. This guide won't show you the finished version — figuring out where it goes and writing it yourself is the exercise.

22. Complete Session 07 End State

What exists right now:

- A typed sample `tasks: Task[]` (six tasks) and `users: User[]` (three people), both plain `const` arrays — genuinely not React state.
- `.map()` rendering the real task list, using arrow-function callbacks and `key={task.id}`.
- `visibleTasks`, derived from status + search + person, all three combining correctly.
- Dynamic `totalCount` / `completedCount` / `pendingCount`, "X of Y tasks shown", and a progress sentence — all derived.
- Real, derived per-person task counts in the people summary, with a stable `key` and intentional styling.
- Status filter buttons and the "All people" person-filter button, both fully wired.
- Named person-filter buttons rendered (real names, real `key`) but not yet wired to anything.
- `type FilterStatus = "all" | "completed" | "pending";`, used to type `currentFilter`.

- `SectionTitle` and `PersonSummary` , inherited from Session 06's homework, now genuine building blocks of the real UI, with their own deliberate styling.

What is explicitly homework, not End (see Sections 19 and 21):

- The named person-filter buttons don't yet change `selectedUserId` when clicked.
- There is no explicit message when a filter/search/person combination matches zero tasks — the list area is currently just blank in that case.
- A single reusable "person with their count" value doesn't exist yet — the people summary and any future consumer of that same relationship would each calculate it separately today.

What is explicitly NOT implemented yet (future sessions, not homework):

- No stateful task collection — `tasks` is still a plain `const` array, not `useState` .
- No Add, Toggle, Delete, or Edit — nothing changes the task collection.
- No callback props anywhere.
- No spread syntax.
- No `useEffect` or data fetching of any kind.
- No `src/types.ts` — the interfaces/type live directly in `App.tsx` .
- No object/props destructuring — every prop is still read as `props.x` .
- No Fragment.
- The Session 06 "Searching for: ..." feedback and Show/Hide Tasks toggle are gone — retired through real, explained, source-changing steps, not silently dropped.

If a syntax or idea isn't named anywhere above as something you built, you have not missed it — it simply isn't part of Session 7's official End. The homework-shaped gaps above are yours to close yourself; this guide deliberately does not show their finished code.

23. Common Mistakes

1. Forgetting `return` inside `.map()`

WRONG:

```
tasks.map((task) => {  
  <TaskItem key={task.id} title={task.title} />;  
});
```

RIGHT:

```
tasks.map((task) => {  
  return <TaskItem key={task.id} title={task.title} />;  
});
```

Why it's wrong: `.map()` needs each call to *produce* a value. Without `return`, the callback produces `undefined` for every item — the list renders nothing, with no error message pointing directly at the missing word.

2. Forgetting `key`

WRONG:

```
{tasks.map((task) => <TaskItem title={task.title} />)}
```

Why it's wrong: React specifically needs a stable identity for each item in a list. Without `key`, React logs a console warning and falls back to guessing based on position — which breaks in surprising ways as soon as anything about the list can change.

3. Putting `key` on the wrong element

WRONG:

```
<TaskItem title={task.title} taskKey={task.id} />
```

Why it's wrong: `key` is not a normal prop your component receives — it's a special instruction to React itself, and it must sit directly on the element `.map()` returns (`<TaskItem key={...} .../>`), never renamed or passed down as an ordinary prop.

4. Using the array index as the key

Why it's a mistake: it "works" today only because nothing yet reorders, adds, or removes items. `task.id` is unique, stable, and already belongs to the data — always prefer it over a position that can shift.

5. Reaching for `.filter()` to render one element per item

WRONG:

```
tasks.filter((task) => <TaskItem key={task.id} title={task.title} />);
```

Why it's wrong: `.filter()` keeps a subset of the *same kind* of item — it never transforms items into something new. Rendering one UI element per array item is exactly what `.map()` is for.

6. Believing `.map()` changes the original array

Why it's a mistake: `.map()` always returns a brand-new array. `tasks.map(...)` never modifies `tasks` itself — you can call it again and again on the same array with no side effects.

7. Storing `visibleTasks` separately instead of deriving it

Why it's a mistake: `visibleTasks` can already be calculated from `tasks` plus the current filters (`currentFilter`, `searchText`, `selectedUserId`) — nothing about it needs to be remembered on its own. If you stored it separately, you would have to manually recompute and re-set it every single time any filter changed, and it is easy to forget one of those spots — exactly the "forgot to update it" bug class state setters exist to prevent. The actual Session End source avoids this entirely by deriving it fresh, every render, the same way you already saw in section 6:

```
const visibleTasks = tasks.filter((task) => {  
    // ...status, search, and person checks...  
});
```

Because this is a plain `const` recalculated on every render (not something stored and set), it can never go stale — there is nothing to forget to update.

8. Making the task collection itself state, today

Why it's wrong (not a compile error, a design mistake): nothing this session ever changes the task collection, so there's no reason yet for `tasks` to be state. Turning `tasks` into state today is genuinely Session 08's first move, once Add/Toggle/Delete/Edit give it a real reason to change.

9. Forgetting one of the three filter conditions

Why it's a mistake: `visibleTasks` requires `matchesFilter` && `matchesSearch` && `matchesPerson` — all three. Dropping any one of them (for example, returning `matchesFilter` && `matchesSearch` only) silently makes person-filtering do nothing, with no error to point at the missing condition.

10. Hardcoding stats or counts

WRONG:

```
<StatCard label="Total Tasks" value={6} />
```

Why it's wrong: the whole point of today is that these numbers come from the data. A hardcoded number stops being correct the instant the sample data changes; `totalCount = tasks.length` never does.

11. Reaching for callback props, spread, or destructuring

Why it's a mistake: none of these are taught yet (callback props and spread are Session 8; destructuring is Session 10). Every prop this session is still read as `props.x`, and no child component reports anything back up to a parent.

24. Glossary

.map() An array method that visits every item and returns a new array built from whatever the callback returns for each one.

key A special prop React uses to give each item in a rendered list a stable identity — not a normal prop your component receives.

arrow function A shorter syntax for writing a function: `(x) => { return x + 1; }` instead of `function (x) { return x + 1; }` — identical behavior.

derived data A value calculated from existing data and state during rendering, never stored in its own `useState`.

visibleTasks The derived subset of `tasks` that currently matches the status, search, and person filters together.

type A keyword that gives a reusable name to a TypeScript type — like `interface`, but also usable for things that aren't object shapes, such as a union of allowed strings.

string-literal union A type built from an exact, closed list of allowed string values, joined with `|` — for example `"all" | "completed" | "pending"`.

FilterStatus This project's own string-literal union type, restricting `currentFilter` to exactly `"all"`, `"completed"`, or `"pending"`.

selectedUserId Numeric state identifying which person's tasks should be shown; `0` means "All people."

empty state A deliberate message shown when a derived list has zero items, instead of leaving a blank area. TaskFlow doesn't have one yet at the end of Session 07 — adding it is homework.

25. Review Questions

1. What's the real difference between `.map()` and `.filter()` ?
 2. Why does `.map()` need a `key` on each item, and why is `task.id` a good choice?
 3. What changed between the normal-function version of the task list and the arrow-function version?
 4. What does it mean for a value to be "derived," and why is `visibleTasks` one?
 5. Why is `tasks` still a plain `const` array at the end of Session 7?
 6. How do the three filter checks inside `visibleTasks` combine?
 7. What does `type FilterStatus = "all" | "completed" | "pending";` actually protect against?
 8. Why does `getOwnerName`'s callback stay a normal function while the task-list `.map()` was rewritten into an arrow function?
 9. Why do the per-person task counts update automatically when the sample data changes?
 10. What does the task list area currently show when a filter/search/person combination matches zero tasks, and why might that be confusing to a real user?
-

26. Answers

1. `.filter()` keeps a subset of the same kind of item — the count can shrink, but no item is transformed. `.map()` never changes the count — it transforms every item into something else (here, a piece of UI).
2. React needs a stable identity for each item so it can tell "same item, still here" from "a new item" when the list re-renders. `task.id` is unique, stable, and already belongs to the data — nothing was invented just for React's benefit.
3. Nothing about the behavior changed — only the function syntax. `function (task) { ... }` became `(task) => { ... }`, the same body, shorter to write.
4. A derived value is calculated fresh, during rendering, from data and state that already exist — never stored separately. `visibleTasks` is derived because it's recalculated from `tasks`, `currentFilter`, `searchText`, and `selectedUserId` every render, rather than being remembered in its own `useState`.
5. Because nothing this session changes the task collection itself — no add, toggle, delete, or edit exists yet. A `const` array is the correct tool for data that's read but never modified.
6. Each of `matchesFilter`, `matchesSearch`, `matchesPerson` is computed independently inside the same `.filter()` callback; a task only survives into `visibleTasks` if `matchesFilter && matchesSearch && matchesPerson` is true — all three at once.

7. It protects `currentFilter` from ever holding a string outside exactly `"all"`, `"completed"`, or `"pending"` — TypeScript rejects anything else at compile time, catching a typo before it becomes a runtime bug.

8. Because rewriting it would change nothing and add risk for no benefit — `getOwnerName`'s `.find()` callback already worked correctly before arrow functions existed in this course. Arrow functions are used for new code written after Section 7, not retroactively applied to code that already works.

9. Because they're never hardcoded — `tasks.filter((task) => task.userId === user.id).length` is recalculated from the real `tasks` array on every render, so any change to the data is reflected automatically.

10. Right now, nothing — the task list area is completely blank, with no rows and no message. A real user could easily mistake that for a broken page rather than a filter combination that happens to match nothing. Fixing this (showing an honest message instead) is this session's STRETCH homework.

27. What You Should Be Able to Explain Now

- ☐ I can explain the real difference between `.map()` and `.filter()`.
 - ☐ I can write a `.map()` call that renders one component per array item, with a correct `key`.
 - ☐ I can explain why `task.id` is a good key and an array index is not.
 - ☐ I can explain the difference between a normal function and an arrow function, and why neither is "more correct."
 - ☐ I can explain what "derived data" means, in my own words.
 - ☐ I can explain why `visibleTasks` is not `useState`, and why `tasks` still isn't either.
 - ☐ I can trace how three independent filter checks combine into one `return` statement.
 - ☐ I can explain what `type FilterStatus = "all" | "completed" | "pending";` protects against.
 - ☐ I can explain why `getOwnerName` was left as a normal function instead of being rewritten as an arrow.
 - ☐ I can explain what currently happens when nothing matches the filters, and why an explicit message would be an improvement (even though I haven't built one yet).
-

28. Not Yet

The following are genuinely real ideas in React and TypeScript — none of them are taught yet, and none of their syntax appears anywhere in this project:

- Making the task collection itself React state — not taught yet.
- Add, Toggle, Delete, Edit — not taught yet.
- Callback props — not taught yet.
- Immutable collection updates — not taught yet.
- Spread syntax — not taught yet.
- `useEffect` — not taught yet.
- API fetching, loading state, error state — not taught yet.
- `src/types.ts` as a separate file — not taught yet.
- Object/props destructuring — not taught yet.
- Fragment — not taught yet.
- State ownership / lifting state — not taught yet.
- A component-architecture refactor — not taught yet.

If every box in Section 27 is honestly checked, you understand Session 7 completely — and you're ready for the one open question your homework's final reflection asks: what would it actually take for a button to change whether a task is completed? That's Session 8.